YADHAVRAO TASGAONKAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

AI&Machine Learning — Module 2

Problem Solving&Search Strategies — Questions with Answers

1. Define State Space Representation. Explain its components with a suitable example.

State Space Representation is a method of representing a problem as a collection of possible states and the actions that can change one state into another. It helps an AI system systematically search for a solution.

Main components:
1. Initial state – the state from which the problem starts.
2. Goal state – the desired final state.
3. Actions/operators – the possible actions that can be performed.
4. State space – all possible states reachable from the initial state.
5. Path/cost – the sequence of actions and, when applicable, the cost of reaching a state.

Example: In a route-finding problem, Mumbai can be the initial state and Pune the goal state. Cities are states, travelling from one city to another is an action, and distance or travel time can be the path cost.

2. Explain Breadth First Search (BFS) with a suitable example.

Breadth First Search (BFS) is an uninformed search algorithm that explores a graph level by level. It first visits the starting node, then all its immediate neighbours, then the neighbours of those nodes.

BFS uses a Queue data structure, which follows FIFO (First In, First Out).

Example:
For the graph A → B, C; B → D, E; C → F, starting from A:
Level 0: A
Level 1: B, C
Level 2: D, E, F

Therefore, the BFS traversal is:
A → B → C → D → E → F

Real-life example: Finding the shortest route in terms of number of stops in an unweighted network. BFS checks all locations one stop away before checking locations two stops away.

BFS is complete and, for an unweighted graph where each step has equal cost, it finds a shortest path.

3. Explain Depth First Search (DFS) with a suitable example.

Depth First Search (DFS) is an uninformed search algorithm that explores one branch as deeply as possible before backtracking and exploring another branch.

DFS can be implemented using:
• Stack data structure
• Recursion

For the graph A → B, C; B → D, E; C → F, if neighbours are considered from left to right, DFS can visit:
A → B → D → E → C → F

Working:
1. Start at A.
2. Visit B.
3. Go deeper to D.
4. D has no unvisited child, so backtrack to B.
5. Visit E.
6. Backtrack to A.
7. Visit C and then F.

Real-life example: Exploring folders on a computer. You can open one folder, then a subfolder, then another subfolder until there is nothing more to explore, and then return to the previous folder.

4. Explain Uniform Cost Search (UCS). How is it different from BFS?

Uniform Cost Search (UCS) expands the node having the lowest total path cost from the starting node. It is useful when different actions have different costs.

UCS uses a priority queue. The priority of a node is based on g(n), where g(n) is the actual cost from the start node to that node.

Example:
Suppose A can reach B with cost 2 and C with cost 5. UCS first considers B because its path cost is 2. If a later path reaches another node with an even lower total cost, UCS gives priority to that path.

Difference between BFS and UCS:
• BFS chooses the shallowest node, usually based on depth.
• UCS chooses the node with the lowest accumulated path cost.
• BFS is suitable when every step has the same cost.
• UCS is suitable when step/action costs are different.
• UCS is optimal when step costs are positive/non-negative under the standard assumptions.

5. What is a heuristic function? Explain its importance in informed search.

A heuristic function, written as h(n), estimates the cost of reaching the goal from the current node n.

It is called an informed search technique because it uses additional problem-specific knowledge to decide which node is more promising.

Example: In a route-finding problem, the straight-line distance from the current city to the destination can be used as a heuristic.

Importance:
1. It guides the search towards the goal.
2. It can reduce the number of nodes explored.
3. It makes search more efficient than blindly exploring all possibilities.
4. It is used by algorithms such as Greedy Best First Search and A* Search.

A good heuristic should provide a useful estimate without requiring excessive computation.

6. Explain Greedy Best First Search with its evaluation function.

Greedy Best First Search is an informed search algorithm that selects the node that appears closest to the goal according to the heuristic value.

Its evaluation function is:
f(n) = h(n)

Here, h(n) is the estimated cost from node n to the goal.

Working:
1. Start from the initial node.
2. Calculate h(n) for available nodes.
3. Select the node with the smallest h(n).
4. Continue until the goal is reached.

Example: If the available nodes have heuristic values B = 7, C = 4 and D = 6, Greedy Best First Search selects C because 4 is the smallest heuristic value.

Advantage: It can reach the goal quickly.
Limitation: It does not consider the actual cost already travelled, so it is not guaranteed to find the least-cost solution.

7. Explain the A* search algorithm. What are the meanings of g(n), h(n), and f(n)?

A* (A-star) Search is an informed search algorithm that combines the actual cost already travelled with an estimate of the remaining cost.

The evaluation function is:
f(n) = g(n) + h(n)

Where:
• g(n) = actual cost from the initial node to node n.

- h(n) = estimated cost from node n to the goal.
- f(n) = estimated total cost of a solution path through node n.

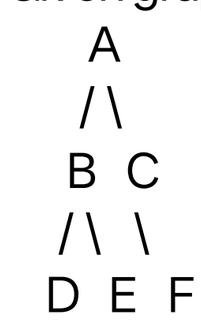
Working:
1. Start from the initial node.
2. Calculate g(n), h(n), and f(n) for candidate nodes.
3. Select the node with the smallest f(n).
4. Expand that node and calculate values for its neighbours.
5. Continue until the goal is selected.

Example: If node B has g(B)=4 and h(B)=6, then f(B)=4+6=10.

A* is generally optimal when an appropriate admissible heuristic is used and the standard conditions for the search are satisfied.

8. For the given graph, perform BFS and DFS traversal starting from node A and write the order of visited nodes.

Given graph:



Assuming neighbours are visited from left to right:

BFS:
Start at A.
Visit A, then its neighbours B and C.
Next visit B's neighbours D and E, followed by C's neighbour F.

BFS order:
A → B → C → D → E → F

DFS:
Start at A and go as deep as possible along the left branch.
A → B → D, then backtrack to B and visit E.
Backtrack to A, then visit C → F.

DFS order:
A → B → D → E → C → F

Note: DFS order can vary if the order in which neighbours are stored/selected is different.

9. Compare Greedy Best First Search and A* Search. Explain why A* considers both the cost already travelled and the estimated remaining cost.

Greedy Best First Search and A* Search are both informed search algorithms, but they use different evaluation functions.

Greedy Best First Search:
f(n) = h(n)
It considers only the estimated remaining cost to the goal.

A* Search:
f(n) = g(n) + h(n)
It considers both the actual cost already travelled and the estimated remaining cost.

Comparison:
• Greedy uses only h(n).
• A* uses g(n) and h(n).
• Greedy may select a path that looks close to the goal but is expensive.
• A* balances the cost already incurred with the estimated remaining cost.
• Greedy is generally faster in some cases but is not guaranteed to find the least-cost path.
• A* can find an optimal solution when its heuristic satisfies the required conditions.

Why A* uses both:
Suppose one path is very close to the goal but has already cost 100, while another path is slightly farther from the goal but has cost 20. Greedy may prefer the first path because its h(n) is small. A* considers both values and can identify the better overall route.

10. Consider a college examination timetable as a Constraint Satisfaction Problem (CSP). Identify the variables, domains and constraints involved.

A Constraint Satisfaction Problem (CSP) consists of variables, domains and constraints.

For a college examination timetable:

Variables:
Each examination can be considered a variable.
Example:
X1 = AI&ML exam
X2 = Database Management exam
X3 = Computer Networks exam
X4 = Mathematics exam

Domains:
The domain contains the possible time slots and rooms for each examination.
Example:
{Monday 10–12, Tuesday 10–12, Wednesday 2–4, Thursday 10–12}

Constraints:
Rules that must be satisfied while creating the timetable.

Examples:
1. A student should not have two examinations at the same time.
2. The same classroom cannot be assigned to two examinations at the same time.
3. A room cannot exceed its seating capacity.
4. An examination must be assigned to an available time slot.
5. Suitable gaps may be required between examinations for the same student group.

Therefore:
Variables = examinations
Domains = available time slots/rooms
Constraints = rules that restrict valid assignments

The aim is to assign a value from the appropriate domain to every variable while satisfying all constraints.

Quick Revision — Important Formulas

BFS: Selects the shallowest/unvisited node using a FIFO queue.

UCS: Selects the node with the lowest path cost g(n).

Greedy Best First Search: f(n) = h(n).

A* Search: f(n) = g(n) + h(n).

g(n): Actual cost from the start node to n.

h(n): Estimated cost from n to the goal.

f(n): Estimated total cost through n.